

Relazione del progetto: WellnessInCloud

Insegnamenti: Programmazione Object Oriented e Basi di Dati

Anno Accademico: 2025-2026

Documento di: Descrizione del Sistema e del Dominio

GitHub repository: <https://github.com/EdoksSW/WellnessInCloud.git>

1. Introduzione e Obiettivi

Il progetto *WellnessInCloud* ha lo scopo di fornire una soluzione gestionale completa e scalabile di palestre e club sportivi. Il progetto punta a digitalizzare ogni fase del rapporto con l'utente: dall'acquisto di abbonamenti o merchandising sul web, fino al monitoraggio fisico degli ingressi. Automatizzando queste procedure e la pianificazione dello staff, il sistema offre agli amministratori una visione d'insieme chiara e centralizzata, ottimizzando i tempi e le risorse della struttura.

2. Presentazione del Dominio

Il dominio scelto riguarda la gestione di una struttura sportiva complessa in tutte le sue sfaccettature. Il sistema deve gestire flussi di dati eterogenei che spaziano dall'anagrafica sensibile (certificati medici) alla logistica avanzata (planning corsi, sale e turni del personale), fino ad arrivare alla gestione finanziaria completa (abbonamenti, ingressi singoli ed e-commerce di prodotti).

2.1 Ambiti del Sistema

- **Utenze e CRM:** Gestione dei permessi per i diversi livelli di staff e monitoraggio costante delle scadenze documentali dei soci per garantire la conformità normativa
- **Planning dei Corsi:** Organizzazione centralizzata delle sale e del calendario lezioni, con un sistema di gestione dei turni pensato per ottimizzare la presenza del personale.
- **Gestione Iscrizioni e Check-in:** Gestione dinamica dei contratti (mensili, tri mensili, annuali o singoli ingressi) con verifica istantanea della regolarità sanitaria dell'utente ai tornelli
- **E-shop e Transazioni:** Integrazione di un carrello per i prodotti fisici e centralizzazione dei flussi di pagamento, permettendo di tracciare in un solo punto ogni movimento finanziario.

3. Requisiti in Linguaggio Naturale

- Il sistema deve gestire diverse tipologie di utenza con permessi differenti: **Amministratore**, **Staff** (istruttori/receptionist) e **Clienti**.
- È obbligatorio caricare un **certificato medico** valido (specificando se agonistico o non agonistico) per consentire l'accesso alla struttura.
- Il planning deve supportare la creazione di **Corsi** generici legati a **Lezioni** specifiche, le quali vengono svolte in determinate **Sale** fisiche della Palestra, mentre lo **Staff** è organizzato in **Turni** di lavoro (mattina, pomeriggio, sera).
- I clienti devono poter gestire le **Iscrizioni**, che porta a 2 **Titoli di Ingresso**, cioè Abbonamenti o Entrate Singole. Il sistema deve includere uno shop online dove il cliente può aggiungere **Prodotti** al **Carrello** e finalizzare un **Ordine**.
- Tutti i pagamenti (che derivino da un'iscrizione o da un ordine e-commerce) devono confluire in un sistema di transazioni unificato.

4. Funzioni ad Alto Livello

Modulo	Funzione	Descrizione Breve
Gestione Utenze	Registrazione e Ruoli	Creazione profili con differenziazione tra Cliente, Staff e Admin.
Controllo Accessi	Validazione Iscrizione	Verifica di Abbonamento/Entrate, incrociata con la validità del Certificato.
Logistica	Gestione Spazi e Turni	Assegnazione Staff ai turni; allocazione di Corsi e Lezioni nelle Sale.
Finance & Shop	Checkout Unificato	Gestione di Carrello, Ordini prodotti e Pagamenti (Iscrizioni e Merchandising).

5. Modello del Dominio (Classi Principali)

Il cuore del sistema è stato strutturato suddividendo le entità in macro-aree logiche. Per massimizzare la riusabilità del codice e la manutenibilità, si è fatto ampio ricorso all'ereditarietà per le gerarchie complesse e alle enumerazioni per la gestione rigorosa degli stati

A. Profilazione e Gerarchia degli Utenti

Il sistema adotta una struttura gerarchica basata sulla classe astratta *Utente*, che funge da radice comune per condividere attributi anagrafici e credenziali di accesso. L'utente ingloba le generalità, cioè il Codice Fiscale come chiave primaria, il nome, il cognome, e-mail, telefono, e data di nascita. Da questa base si diramano tre specializzazioni distinte:

- **Cliente**: il destinatario dei servizi, i cui flussi operativi si intrecciano con le prenotazioni, la gestione sanitaria dei certificati e gli acquisti (sia di iscrizioni che di prodotti fisici). Vengono memorizzati il suo indirizzo, il numero civico, il CAP, la carta fedeltà e lo "stato account" che determina se l'abbonamento è attivo, bloccato (da pagare) o in revisione (attivo ma con mancanza di Certificato Medico), come specificato nell'apposita **Enumeration**.
- **Staff**: rappresenta il dipendente della palestra, diviso in istruttori e receptionist. La sua operatività è definita da un Ruolo specifico (gestito tramite Enum) e dalla sua collocazione all'interno dei calendari dei corsi e dei turni lavorativi. Per la ricezione dello stipendio consideriamo anche l'iban come attributo.
- **Admin**: il supervisore con privilegi elevati, come la pianificazione dei corsi e la gestione del prezzario.

B. Infrastruttura Logistica e Planning

L'organizzazione delle attività è mappata riflettendo la struttura fisica dei centri. Come classe cardine di questa compartimentazione prendiamo le singole Sale. La programmazione segue una logica a due livelli:

1. Il **Corso** definisce l'entità astratta dell'attività (ad esempio "Pesistica" o "Calisthenics"), collegandola a un istruttore dello staff e a una sede.
2. La **Lezione** rappresenta l'istanza temporale concreta a cui i soci si iscrivono tramite la classe **Prenotazione**, la quale può essere Valida, Disdetta o in Lista di attesa a seconda del suo stato di accettazione, come specificato nell' Enum. Il coordinamento del personale è garantito dalla classe **Turno**, che categorizza gli orari di inizio e fine lezione, oltre alla data e l'identificatore univoco del turno specifico da istanziare.
3. La **Sala** permette invece di compartimentare le varie lezioni per una gestione coordinata di corsi diversi. Ogni sala ha un nome e una capienza massima da rispettare obbligatoriamente.

C. Controllo Accessi e Tipologie di Servizio

La sicurezza e la flessibilità commerciale sono gestite tramite un sistema di validazione dei titoli di ingresso. Il *CertificatoMedico* è il requisito vincolante per ogni attività, con una distinzione esplicita tra tipologia agonistica e non agonistica (gestita con Enum), l'indicazione della data di emissione e di scadenza. Le modalità sono collegate all'entità *Iscrizione*, composta dalle generalità del cliente, il tipo di abbonamento e le informazioni del pagamento. Le tipologie di accesso sono unificate nell'entità *TitoloIngresso*, grazie all'attributo tipo (gestito tramite l'Enum Tipo_Tit) il sistema distingue tra abbonamenti (caratterizzati da una durata_mesi) e iscrizioni singole, mantenendo comuni i prezzi e la data di inizio.

D. Modulo E-Commerce e Vendite

Per la vendita di prodotti fisici è stato implementato un modulo di shopping semplificato. I prodotti sono catalogati nell'entità *Categoria* a seconda delle sue caratteristiche e possono essere gestiti temporaneamente nel *Carrello* dell'utente. In quest'entità viene riportata la lista dei prodotti e il totale da pagare per uno specifico Cliente. Una volta finalizzato l'acquisto, il sistema genera un *Ordine*, monitorandone lo stato (in revisione, approvato, in spedizione, spedito e consegnato) e i costi totali. L'entità *Pagamento* permette al sistema di trattare in modo uniforme le transazioni derivanti dalle iscrizioni sportive e quelle originate dagli ordini e-commerce, questo è possibile perché l'entità Pagamento mette in relazione proprio l'Ordine(per lo shop) e l'iscrizione (per la palestra) con la transazione economica effettiva. Questo approccio centralizzato permette di tracciare ogni movimento economico verso un unico modulo di reportistica, indipendentemente dalla natura del bene acquistato.

7. Scelte Progettuali Preliminari

Il sistema verrà sviluppato in linguaggio Java seguendo fedelmente il pattern **BCE (Boundary-Control-Entity)**, mentre lo strato di persistenza sarà gestito tramite il pattern **DAO (Data Access Object)** interfacciato con un database relazionale (PostgreSQL). Per la gestione del ciclo di vita del software e delle dipendenze verrà utilizzato **Apache Maven**.

Repository GitHub:[EdoksSW/WellnessInCloud](https://github.com/EdoksSW/WellnessInCloud)